



Security Assessment Final Report



M0 Stellar Minter Gateway

May 2026

Prepared for M0

Table of Contents

Project Summary	3
Project Scope	3
Project Overview	3
Protocol Overview	3
Assessment Methodology	5
Threat and Security Overview	6
Findings Summary	10
Severity Matrix	10
Detailed Findings	11
Low Severity Issues	12
L-01: decrease_both_accumulators / burn are missing a `total_supply` underflow check	12
L-02: set_rate isn't protected by the pausing mechanism	14
L-03: The force_transfer function is pause-gated, contradicting its role as a compliance primitive	16
L-04: Wrapper accumulators desync from real circulating supply on send-to-issuer and reconciliation is pause-blocked	17
L-05: Small burn amounts (1 unit) would be zeroed out during principal calculations due to rounding down	19
Informational Severity Issues	21
I-01: Main admin setter function should implement a two-step transfer pattern	21
Disclaimer	22
About Certora	22

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
MO Stellar Minter Gateway	https://github.com/mO-foundation/stellar-minter-gateway	Obadee6 (initial) 95ce1f0 (fix)	Stellar

Project Overview

This document describes the security review of **MO Stellar Minter Gateway**. The work was undertaken from **May 4th, 2026** to **May 7th, 2026**.

The following contract list is included in our scope:

`contracts/mintergateway/src/*`

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

MGUSD Minter Gateway is a Soroban smart contract on the Stellar network that administers a Stellar Asset Contract (SAC) wrapping a USD-pegged stablecoin issued by MO. The wrapper does not hold user balances — those live on the SAC at the protocol level — but mediates all supply changes and serves as the SAC's admin.

The contract supports four core flows:

- **Minting** is performed by the Bridge role, which receives fiat off-chain and mints corresponding SAC tokens to whitelisted recipient accounts.
- **Burning** is performed by the Bridge to redeem SAC tokens for fiat, using the SAC's **clawback** operation.

- **Yield accrual** uses a continuous-compounding index model ($\text{currentIndex} = \text{latestIndex} \times e^{(\text{rate} \times \Delta t)}$) implemented via a 4-term Taylor series. Yield accrues on `total_principal` only and is paid to a designated yield recipient via `claim_yield`, which mints fresh tokens against earned interest.
- **Compliance enforcement** uses the SAC's `AUTH_REQUIRED` flag combined with a wrapper-level allowlist. Block and unblock operators (typically Crossmint) authorize accounts via `set_authorized`. A separate Forced Transfer Manager role can clawback and re-mint tokens between accounts (e.g., for sanctions enforcement or court-ordered seizures).

Authorization is split across eight roles — Admin, Minter, Yield Recipient Manager, Yield Recipient (passive), Forced Transfer Manager, Block operator, Unblock operator, and Pauser. Admin is **not** a super-role: it can rotate other roles and reconcile accumulators, but cannot directly mint, burn, freeze, or pause without first granting itself the relevant role. The contract is upgradeable via WASM hash replacement, gated by the Admin role.

The audit covered the wrapper contract source (`contracts/mintergateway/src/`, ~1,025 LoC across 11 files).

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

Threat analysis for the MGUSD wrapper — a Soroban contract that administers a Stellar Asset Contract (SAC) and accrues continuous yield. Covers `contract.rs`, `yield_state.rs`, `continuous_index.rs`, `roles.rs`, `admin.rs`, `sac_token.rs`.

1. Yield Accounting & Index Integrity

Affected Components: `yield_state.rs`, `continuous_index.rs`, `contract.rs` (`mint`, `burn`, `claim_yield`, `set_interest_rate`)

Risk: Yield accrues via a 4-term Taylor expansion of $e^{(\text{rate} \times \Delta t)}$ and two accumulators (`total_principal` in PV, `total_supply` in nominal). Any drift in the index or the PV conversion can produce phantom yield or lock real yield.

What we are verifying:

- Index is monotonically non-decreasing while `rate > 0`; frozen when `rate = 0`.
- `update_index` runs before every state mutation that depends on the index.
- Mint floors PV, burn ceils PV — both directions protocol-favorable.
- Accrued yield is clamped at zero (never negative from rounding residue).
- Yield accrues on principal only — claimed yield does not compound.
- Rate changes finalize yield at the old rate first.

2. Supply Accumulator ↔ SAC Desynchronization

Affected Components: `contract.rs` (`reconcile_burn`, `burn`), `yield_state.rs` (`decrease_both_accumulators`)

Risk: Holders can destroy tokens directly via SAC `transfer()` to the issuer, bypassing the wrapper's accumulators. Without reconciliation, the wrapper earns yield against phantom principal.

What we are verifying:

- `reconcile_burn` is admin-only, remains callable during a pause, and only mutates wrapper bookkeeping.
- `total_supply` is checked before `total_principal` on every burn path (typed `BurnExceedsSupply`).
- Accumulators never go negative even after the index grows past `INDEX_SCALE`.

3. Role-Based Access Control

Affected Components: `roles.rs`, `admin.rs`, `contract.rs`

Risk: Eight roles (`admin`, `minter`, `yield_recipient`, `yield_recipient_manager`, `forced_transfer_manager`, `block_operator`, `unblock_operator`, `pauser`). `admin` rotates the others; semi-trusted roles (`minter`, `forced_transfer_manager`) are in-scope.

What we are verifying:

- Every state-modifying entrypoint enforces exactly one role check.
- `admin` is not a super-role — cannot pause, claim, force-transfer, or block.
- Operator sets are idempotent and don't accumulate duplicates.
- `transfer_sac_admin` is one-way: SAC authority is permanently lost on the contract side.

4. Pause Mechanism Coverage

Affected Components: `contract.rs` (Pausable impl + every `when_not_paused` site)

Risk: Pause must freeze the right operations: financial state changes blocked, but compliance and reconciliation primitives must remain live because the threats they address can fire during a pause.

What we are verifying:

- `mint`, `burn`, `claim_yield`, `set_interest_rate` revert when paused.
- `force_transfer`, `block_user`, `unblock_user`, `reconcile_burn` remain callable when paused.
- Multi-pauser membership grants the role; no pauser is privileged over another.

5. Compliance Primitives & SAC Authorization

Affected Components: `contract.rs` (`block_user`, `unblock_user`, batch variants, `force_transfer`, `blocked`)

Risk: Compliance flows toggle SAC authorization and force-move tokens. The wrapper/SAC boundary traps (e.g., `authorized()` traps with no trustline) must be normalized into typed errors. Force-transfer must not touch accumulators.

What we are verifying:

- `blocked()` returns true on any non-Ok(true) outcome from `try_authorized`.
- `force_transfer` preflights the destination before clawing back the source; does not modify accumulators.
- Mint and `claim_yield` preflight destination authorization.

- Batch block/unblock capped at `MAX_BATCH_SIZE = 40`, one event per user.

6. Initialization, Upgrade & TTL Safety

Affected Components: `contract.rs` (`__constructor`, `upgrade`, `transfer_sac_admin`), `storage_types.rs`

Risk: The wrapper is constructor-initialized once, owns the SAC admin key, and is admin-upgradeable. Soroban storage expires under TTL.

What we are verifying:

- `__constructor` reverts on re-initialization.
- Every public function extends instance TTL.
- `upgrade()` is admin-only; the new WASM hash must already be uploaded (off-chain verified).
- `transfer_sac_admin` is irreversible — once executed, `mint/burn/clawback` are permanently lost.
- Admin transfer is single-step by design.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	–	–	–
High	–	–	–
Medium	–	–	–
Low	5	5	5
Informational	1	1	–
Total	6	6	5

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
L-01	decrease_both_accumulators / burn are missing a `total_supply` underflow check	Low	Fixed
L-02	set_rate isn't protected by the pausing mechanism	Low	Fixed
L-03	The force_transfer function is pause-gated, contradicting its role as a compliance primitive	Low	Fixed
L-04	Wrapper accumulators desync from real circulating supply on send-to-issuer and reconciliation is pause-blocked	Low	Fixed
L-05	Small burn amounts (1 unit) would be zeroed out during principal calculations due to rounding down	Low	Fixed
I-01	Main admin setter function should implement a two-step transfer pattern	Informational	Acknowledged

Low Severity Issues

L-01: `decrease_both_accumulators` / `burn` are missing a ``total_supply`` underflow check

Severity: Low	Impact: Low	Likelihood: Low
Files: contracts/mintergateway/src /yield_state.rs	Status: Fixed	

Description:

During `reconcile_burn` there's a check to make sure that the amount being burned isn't greater than the `total_supply`:

```
if amount > get_total_supply(&e) {  
    return Err(MinterGatewayError::BurnExceedsSupply);  
}
```

This is because the minter is responsible for burning an equivalent amount of tokens as the ones sent to the issuer address, and the action isn't "atomic".

The problem is that there is a missing guard/check on other burn paths, either in `burn` or a one in `decrease_both_accumulators`. Considering that the minter role can theoretically burn a greater amount than the one sent to the issuer, a situation might arise in which a legitimate burn is more than the `total_supply`.

Recommendations:

Include a `total_supply` underflow check in `burn` or only one in `decrease_both_accumulators` since both burn paths go through it.

Customer Response:

Fixed in [65a000a](#).

Moved the `amount > total_supply` guard into `decrease_both_accumulators` so both `burn` and `reconcile_burn` share a typed `BurnExceedsSupply` failure, and dropped the now-redundant duplicate in `reconcile_burn`. The PV-based principal check is insufficient once the index grows above `INDEX_SCALE` — PV shrinks below nominal, so a burn exceeding `total_supply` could pass the principal guard and reach the supply mutation. The existing `reconcile` test still exercises the same error code; the burn path test was upgraded from a generic `is_err` to a typed `BurnExceedsSupply` assertion.

Fix Review:

Fix confirmed.

L-02: set_rate isn't protected by the pausing mechanism

Severity: Low	Impact: Low	Likelihood: Low
Files: contracts/mintergateway/src /contract.rs	Status: Fixed	

Description:

During an event in which the pausing mechanism has been activated, the functions for which the `minter` is responsible like `mint/burn` are blocked by the pause check.

```
pub fn burn(  
    e: Env,  
    caller: Address,  
    from: Address,  
    amount: i128,  
) -> Result<(), MinterGatewayError> {  
    pausable::when_not_paused(&e);
```

The problem is that `set_rate` which is also a function called by the `minter_role` which will affect the amount of yield accumulated during a pausing period.

Considering that yield is accumulated during a period when the protocol is paused, `minter` can adjust it after a pause has been initiated, they can adjust it to be any extreme value from 0 to 100%.

This could be especially problematic if a `minter` role is compromised.

Recommendations:

There could be multiple suggestions on how to mitigate risks related to yield accumulation during a pausing event.

- Include a `when_not_paused` check in the `set_rate` function.
- Make the `set_rate` function callable by the `admin`, rather than the `minter`.

- Or include a `when_not_paused` check, and automatically switch the yield to 0, after a pause has been initiated.

Customer Response:

Fixed in [287a29e](#).

Added `pausable::when_not_paused(&e)` as the first line of `set_rate`, matching every other state-modifying entrypoint (`mint`, `burn`, `reconcile_burn`, `force_transfer`, `claim_yield`). Closes the inconsistency where a Minter (or a compromised Minter key) could crystallize the index and mutate `rate_bps` during an emergency pause, producing an outsized accrual on unpauses. We kept the function on the Minter role rather than moving it to Admin — rate management is part of normal yield ops and Admin is intentionally not a super-role. Regression test added to `test_pausable`.

Fix Review:

Fix confirmed.

L-03: The `force_transfer` function is pause-gated, contradicting its role as a compliance primitive

Severity: Low	Impact: Low	Likelihood: Low
Files: contracts/mintergateway/src /contracts.rs	Status: Fixed	

Description:

`force_transfer` is documented as a compliance primitive ([docs/MGUSD-IMPLEMENTATION-OVERVIEW.md](#):§5). The same document states at line 389 that compliance operations remain accessible during a pause. The implementation contradicts this: `force_transfer` is gated by `pausable::when_not_paused` ([contract.rs](#):467), while `block_user/unblock_user` are not.

Recommendations:

Remove `pausable::when_not_paused(&e)` from `force_transfer`.

Customer Response:

Fixed in [53eacbf](#).

Removed `pausable::when_not_paused` from `force_transfer` so compliance operations (sanctions enforcement, court-ordered seizures) remain accessible during a pause, consistent with `block_user` / `unblock_user`. Updated the pause doc comment and the pause-effects table in [MGUSD-IMPLEMENTATION-OVERVIEW.md](#) to list `force_transfer` alongside the other compliance ops, and flipped the `test_pausable` assertion to pin the un-gated behavior so a future regression can't silently re-add the gate.

Fix Review:

Fix confirmed.

L-04: Wrapper accumulators desync from real circulating supply on send-to-issuer and reconciliation is pause-blocked

Severity: Low	Impact: Medium	Likelihood: Low
Files: contracts/mintergateway/src /contract.rs	Status: Fixed	

Description:

The wrapper's accumulators (`total_principal`, `total_supply`) are updated only when mint/burn is routed through the wrapper. Stellar's classic protocol allows any authorized holder to send tokens directly to the asset issuer via `PaymentOp` or SAC `transfer()`. During that operation the received amount is destroyed at the protocol level. This path bypasses the wrapper, leaving the accumulators unchanged while the real circulating supply decreases.

After such destruction, the wrapper continues to treat the destroyed amount as live yield-earning principal. Each `claim_yield` mints fresh SAC tokens to the recipient based on the yield computed against this phantom principal. At 5% continuous rate, phantom yield accrues at ~5% of the destroyed amount per year and continues until admin invokes `reconcile_burn`.

`reconcile_burn` is the only on-chain remediation, and it is gated by `pausable::when_not_paused` (`contract.rs:406`). The destruction path is not gated by the wrapper's pause flag and it executes outside wrapper code entirely. During a pause, holders can keep destroying tokens, but admin cannot reconcile. Divergence grows unboundedly until the pause ends. An unrelated incident response (e.g., Bridge compromise) thus disables the contract's only defense against this issue while the issue itself continues to compound.

The README acknowledges the destruction behavior in [Note 2 – Send-to-Issuer Bypasses Yield Accrual](#) and points to `reconcile_burn` as the mitigation, but does not address the pause asymmetry.

Recommendations:

Remove `pausable::when_not_paused(&e)` from `reconcile_burn`. The function is admin-only, modifies wrapper bookkeeping only, and does not interact with the SAC. Pause gating doesn't provide security benefits and removes the only remediation path when it is most needed.

Customer Response:

Fixed in [95ce1f0](#).

Dropped `pausable::when_not_paused` from `reconcile_burn`, updated the `Pausable` impl comment and the pause-effects table in `MGUSD-IMPLEMENTATION-OVERVIEW.md`, flipped the `pausable` test to assert reconciliation works while paused, and added an end-to-end test covering the pause-then-send-to-issuer sequence.

`reconcile_burn` is admin-only and mutates wrapper bookkeeping only (no SAC interaction), so pause gating provided no security benefit while removing the only on-chain remediation for SAC-layer send-to-issuer destruction that continues during a pause.

Note: the underlying off-path send-to-issuer behavior itself remains an accepted operational risk — it is a treasury-only path, exercised in lockstep with an equivalent reduction of the off-chain yield source and documented in the deploy/operations runbook. We prefer to keep the contract surface narrow over introducing a split-attribution variant; this PR addresses Certora's specific concern that the only on-chain remediation was disabled during pause.

Fix Review:

Fix confirmed.

L-05: Small burn amounts (1 unit) would be zeroed out during principal calculations due to rounding down

Severity: Low	Impact: Medium	Likelihood: Low
Files: contracts/mintergateway/src /contract.rs contracts/mintergateway/src /yield_state.rs	Status: Fixed	

Description:

`mint` / `burn` paths initiate the `increase_both_accumulators` and `decrease_both_accumulators` functions which will calculate the principal value of the mint/burn amount based on the current index.

```
pub fn decrease_both_accumulators(env: &Env, amount: i128) -> Result<(),
MinterGatewayError> {
    let mut state = read_yield_state(env);
    let pv_amount = amount
        .fixed_mul_floor(INDEX_SCALE, state.latest_index)
        .unwrap();
    if pv_amount > state.total_principal {
        return Err(MinterGatewayError::BurnExceedsPrincipal);
    }
    state.total_principal =
state.total_principal.checked_sub(pv_amount).unwrap();
    state.total_supply = state.total_supply.checked_sub(amount).unwrap();
}
```

The problem is that any amount > 0 is allowed to be minted/burned. During minting this is done in favor of the protocol, so there's no real risk of any over-calculation on the accumulated yield.

On the other hand during burns, this is also rounded-down, but in this case it isn't in favor of the protocol, as the `pv_amount` can be zeroed-out while the `total_supply` is decreased, leading to the inflation of the calculated yield.

A simple scenario:

- `amount = 1;`
- `latest_index = 1.01`
- `pv_amount = 1 * 1 / 1.01 = 0` (Simplified from 12 decimals, for the purpose of the example)

With this, the `total_supply` will be decreased, while the `total_principal` remains the same, leading to a slight inflation of the principal.

During yield calculations this could lead to more yield being calculated.

```
let total_supply_with_yield = state
    .total_principal
    .fixed_mul_floor(current_index, INDEX_SCALE)
    .unwrap();

(total_supply_with_yield - state.total_supply).max(0)
```

Although the cost of the attack would likely trump any newly generated yield, this still remains a grieving vector which can be used to inflate the generated yield of the protocol.

Recommendations:

Round up when calculating the PV burn amount.

Customer Response:

Fixed in [66e459a](#).

Switched `decrease_both_accumulators` from `fixed_mul_floor` to `fixed_mul_ceil` when computing PV from a nominal burn amount. Mint still floors (under-records principal → protocol-favorable); burn now ceils (over-removes principal → also protocol-favorable). Existing tests that pinned exact floor PV were updated via a shared `pv_ceil` helper, and a regression test runs 1,000 single-stroop burns and asserts principal drops by ≥1 PV unit per burn — fails with principal dropped by 0 under the old floor path.

Fix Review:

Fix confirmed.

Informational Severity Issues

I-01: Main admin setter function should implement a two-step transfer pattern

Description:

The main admin setter function doesn't implement a "two-step" transfer pattern which can be detrimental to the whole protocol's authority model if the admin rights are transferred to an invalid/wrong address.

Recommendations:

Make sure that the current admin can "propose" the new admin address, while the new admin address should accept it, and only then the change should take place.

Customer Response:

Acknowledged.

RISK ACCEPTED. The wrapper Admin role is held by a Stellar multisig with N-of-M signers and an off-chain runbook for admin transfers.

Single-step is acceptable because (a) admin auth already requires multisig consensus, so a "single mistaken or malicious tx" requires signer-layer collusion, (b) production transfers are rehearsed against the destination on testnet first, and (c) layering an on-chain pending/accept pattern on top of multisig adds complexity without materially improving the security posture.

We may revisit in a future contract revision if the threat model changes.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.